

DOCUMENTO DE ARQUITECTURA TÉCNICA

Sistema Integrador de Reservaciones para Community Tours Sian Ka'an

Versión: 1.0

Estado: Arquitectura propuesta para implementación

Base documental: Documento Maestro del Proyecto v2.0

1. Introducción

El presente documento define la arquitectura técnica del **Sistema Integrador de Reservaciones para Community Tours Sian Ka'an**, estableciendo las tecnologías, componentes, capas, mecanismos de comunicación y criterios técnicos que servirán como base para el desarrollo del sistema.

El sistema será una aplicación de escritorio orientada al personal encargado de la gestión de reservaciones. Su objetivo técnico es centralizar en una misma solución la información relacionada con reservaciones provenientes de diferentes plataformas y permitir su consulta, administración, seguimiento y utilización para la generación del Daily.

La arquitectura busca facilitar el mantenimiento, escalabilidad, seguridad, trazabilidad y futura integración con servicios externos.

2. Objetivo de la arquitectura

Definir una arquitectura modular que permita separar claramente:

- La interfaz de usuario.
- La lógica de negocio.
- El acceso y persistencia de datos.
- La autenticación y autorización.
- La auditoría de operaciones.
- Las integraciones con plataformas externas.
- Las funciones propias de la aplicación de escritorio.

La separación de responsabilidades permitirá modificar o ampliar componentes del sistema reduciendo el impacto sobre el resto de la aplicación.

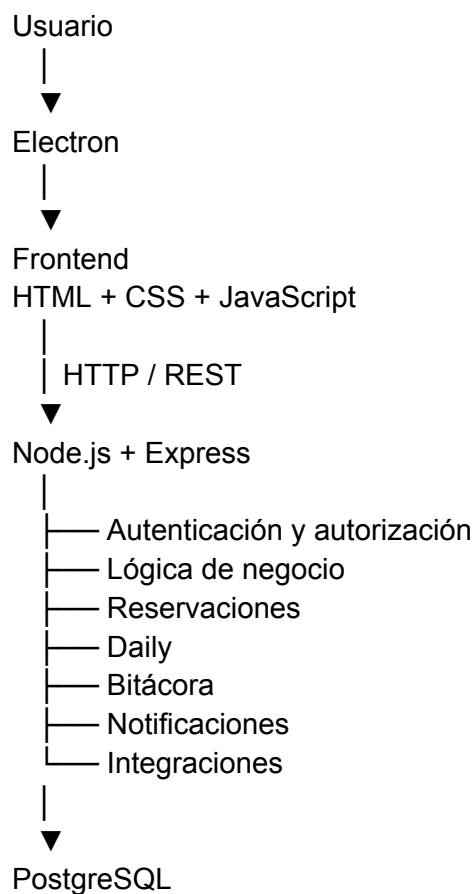
3. Tipo de arquitectura

El sistema utilizará una **arquitectura por capas con comunicación cliente-servidor mediante una API REST**.

La aplicación estará compuesta principalmente por:

1. Aplicación de escritorio.
2. Capa de presentación.
3. API / Backend.
4. Lógica de negocio.
5. Capa de acceso a datos.
6. Base de datos.
7. Capa de integración con servicios externos.

La estructura general será:



4. Tecnologías seleccionadas

4.1 Aplicación de escritorio

Tecnología: Electron

Electron funcionará como contenedor de escritorio para la aplicación y permitirá distribuir el sistema como una aplicación instalable en los equipos de la empresa.

Su responsabilidad será proporcionar el entorno de ejecución de escritorio y cargar la interfaz del sistema.

4.2 Frontend

Tecnologías:

- HTML5
- CSS3
- JavaScript

El frontend será responsable de presentar la información y permitir la interacción entre los usuarios y el sistema.

Las interfaces diseñadas mediante **Google Stitch** serán utilizadas como referencia para la implementación visual.

Entre las pantallas principales se encuentran:

- Login.
- Dashboard.
- Reservaciones.
- Detalle de reservación.
- Nueva reservación.
- Editar reservación.
- Daily.
- Daily Preview.
- Historial.
- Bitácora.
- Detalle de acción.
- Notificaciones.
- Usuarios.

El frontend no realizará conexiones directas con PostgreSQL.

Toda operación sobre los datos deberá realizarse mediante el backend.

5. Backend

Tecnologías:

- Node.js
- Express

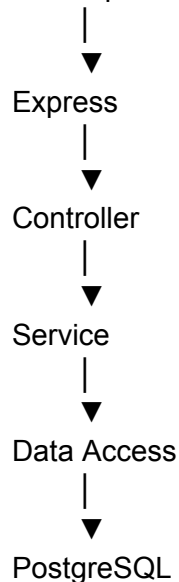
El backend será responsable de centralizar la lógica del sistema y actuar como intermediario entre el frontend, PostgreSQL y los servicios externos.

La comunicación entre frontend y backend se realizará mediante solicitudes HTTP a una API REST.

Ejemplo:

Frontend

GET /api/reservaciones



6. Organización interna del backend

El backend se dividirá inicialmente en los siguientes componentes.

Routes

Definen los endpoints disponibles en la API.

Ejemplos:

/api/auth
/api/usuarios

/api/reservaciones
/api/tours
/api/plataformas
/api/daily
/api/bitacora
/api/notificaciones

Controllers

Reciben las solicitudes provenientes de las rutas, validan la información necesaria y coordinan la ejecución de las operaciones.

Services

Contienen la lógica de negocio principal.

Por ejemplo:

- Crear reservación.
- Modificar reservación.
- Cancelar reservación.
- Consultar reservaciones.
- Generar información del Daily.
- Registrar operaciones en bitácora.
- Procesar información recibida desde plataformas externas.

Data Access

Será responsable de realizar las operaciones necesarias sobre PostgreSQL.

Esto evitará colocar consultas SQL directamente en las interfaces o distribuirlas innecesariamente por la aplicación.

Middleware

Permitirá ejecutar procesos comunes antes de completar determinadas solicitudes.

Principalmente:

- Autenticación.
- Autorización por roles.
- Validación.
- Manejo centralizado de errores.

7. Base de datos

Sistema Gestor de Base de Datos: PostgreSQL.

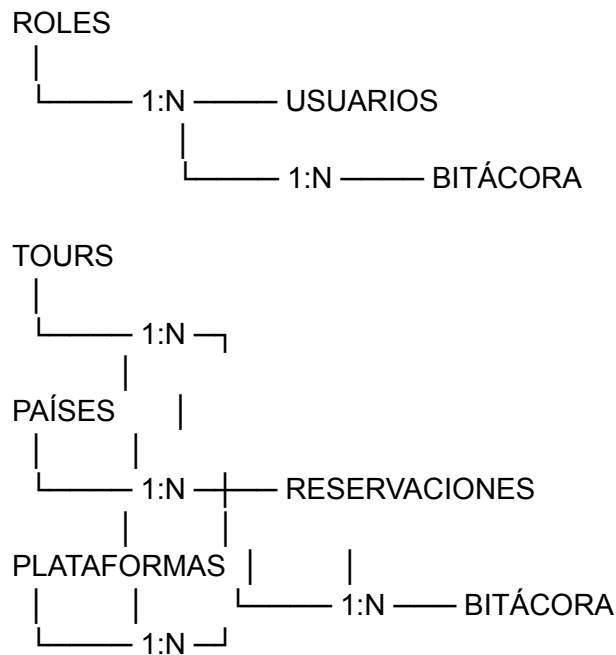
PostgreSQL almacenará permanentemente la información operativa del sistema.

El modelo parte del DER previamente definido para el proyecto.

Las entidades principales serán:

- Roles.
- Usuarios.
- Tours.
- Países.
- Plataformas.
- Reservaciones.
- Bitácora.

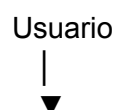
Relaciones principales:

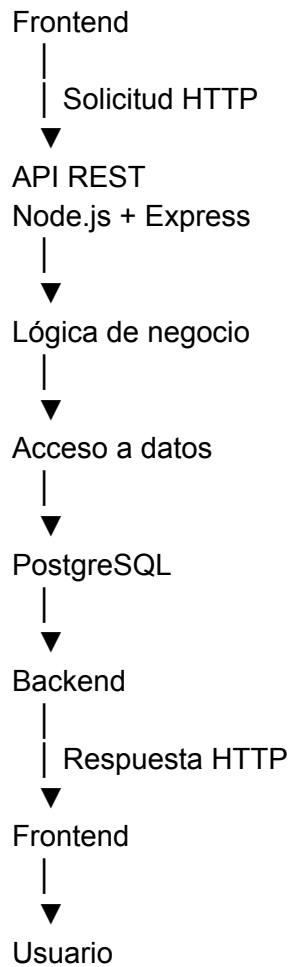


El modelo físico será adaptado específicamente a los tipos de datos, restricciones, claves y características de PostgreSQL.

8. Comunicación entre componentes

La comunicación principal seguirá el siguiente flujo:





El frontend no tendrá acceso directo a las credenciales de PostgreSQL.

9. Autenticación y autorización

El sistema contará con autenticación de usuarios.

Inicialmente existen dos perfiles funcionales:

Administrador / Reservasiones

Tendrá permisos para realizar operaciones como:

- Consultar reservasiones.
- Registrar reservasiones externas.
- Modificar reservasiones.
- Consultar historial.
- Consultar bitácora.
- Generar Daily.
- Exportar Daily.

- Imprimir.
- Consultar notificaciones.

Usuario de consulta

Podrá:

- Iniciar sesión.
- Consultar reservaciones.
- Buscar reservaciones.
- Consultar historial.

No tendrá permisos para modificar información.

La autorización deberá comprobarse en el backend y no depender únicamente de que determinadas opciones estén ocultas en la interfaz.

10. Seguridad

La arquitectura deberá considerar como mínimo:

- Contraseñas almacenadas mediante hash seguro.
- Credenciales de PostgreSQL fuera del código fuente.
- Variables de entorno para información sensible de configuración.
- Validación de datos recibidos por la API.
- Consultas parametrizadas para reducir riesgos de inyección SQL.
- Autorización basada en roles.
- Registro de operaciones relevantes mediante la bitácora.
- Manejo centralizado de errores.
- Restricción de operaciones según los permisos del usuario.

Las credenciales de APIs externas tampoco deberán almacenarse directamente dentro del código fuente.

11. Bitácora y trazabilidad

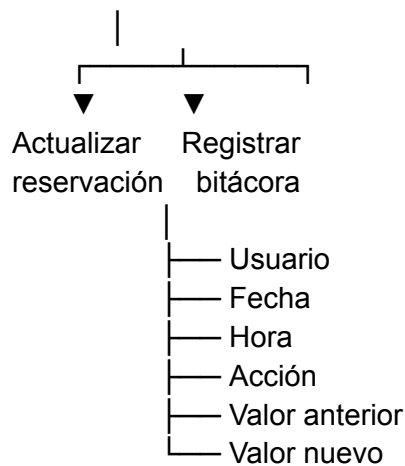
Las modificaciones relevantes deberán pasar por el backend para permitir su registro.

Flujo general:

Usuario modifica reservación



Backend



Esto permitirá conocer quién realizó una modificación, cuándo se realizó y qué información cambió.

12. Integraciones externas

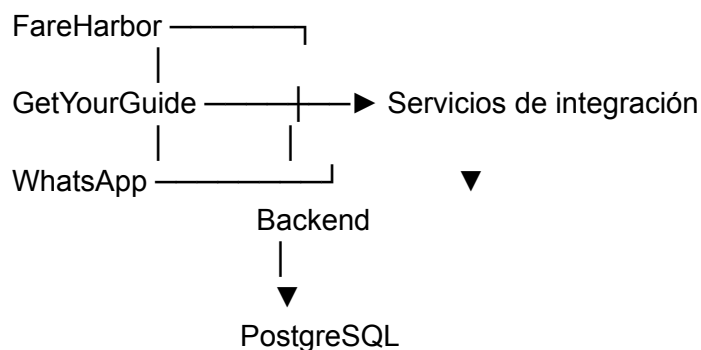
La arquitectura deberá permitir integrar fuentes externas de reservaciones.

Las plataformas contempladas son:

- FareHarbor.
- GetYourGuide.
- WhatsApp Business.
- Reservaciones externas.

La integración automática estará aislada de la lógica principal mediante servicios específicos.

Conceptualmente:



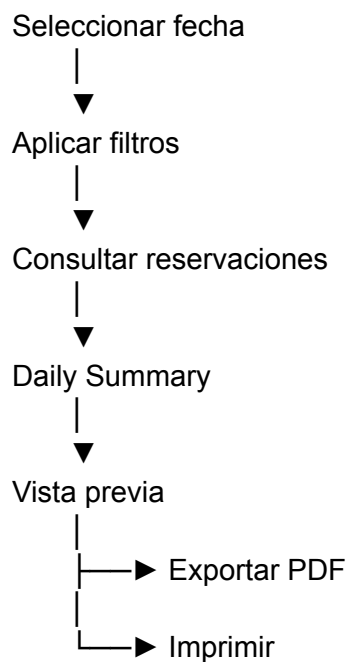
La implementación concreta de cada integración dependerá de la disponibilidad de APIs, permisos, credenciales y mecanismos proporcionados por cada plataforma.

Las reservaciones externas o aquellas que no puedan obtenerse automáticamente podrán registrarse mediante los mecanismos manuales definidos por el sistema.

13. Módulo Daily

El Daily utilizará las reservaciones almacenadas en PostgreSQL para obtener la información correspondiente a una fecha determinada.

Flujo:



La generación del Daily deberá realizarse a partir de la información centralizada por el sistema.

14. Notificaciones

El sistema contará con un módulo de notificaciones para presentar eventos relevantes relacionados con la operación.

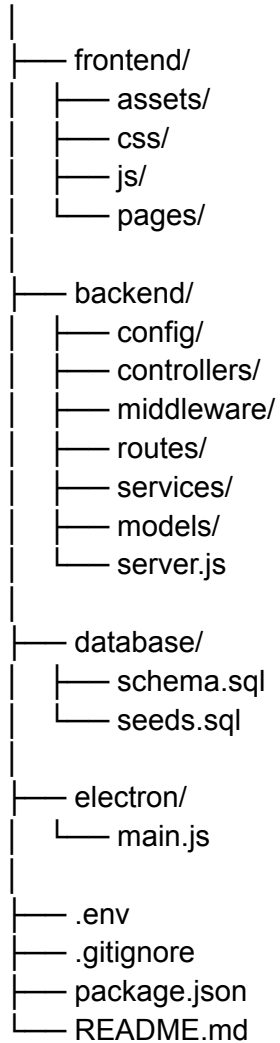
La arquitectura deberá permitir almacenar y consultar las notificaciones y manejar estados como leído/no leído.

La definición técnica detallada de qué eventos generan automáticamente una notificación será especificada durante la implementación del módulo.

15. Estructura preliminar del proyecto

La organización inicial propuesta será:

sistema-integrador-reservaciones/



La estructura podrá ajustarse durante la creación inicial del proyecto si se detecta una separación técnica más adecuada.

16. Entorno de desarrollo

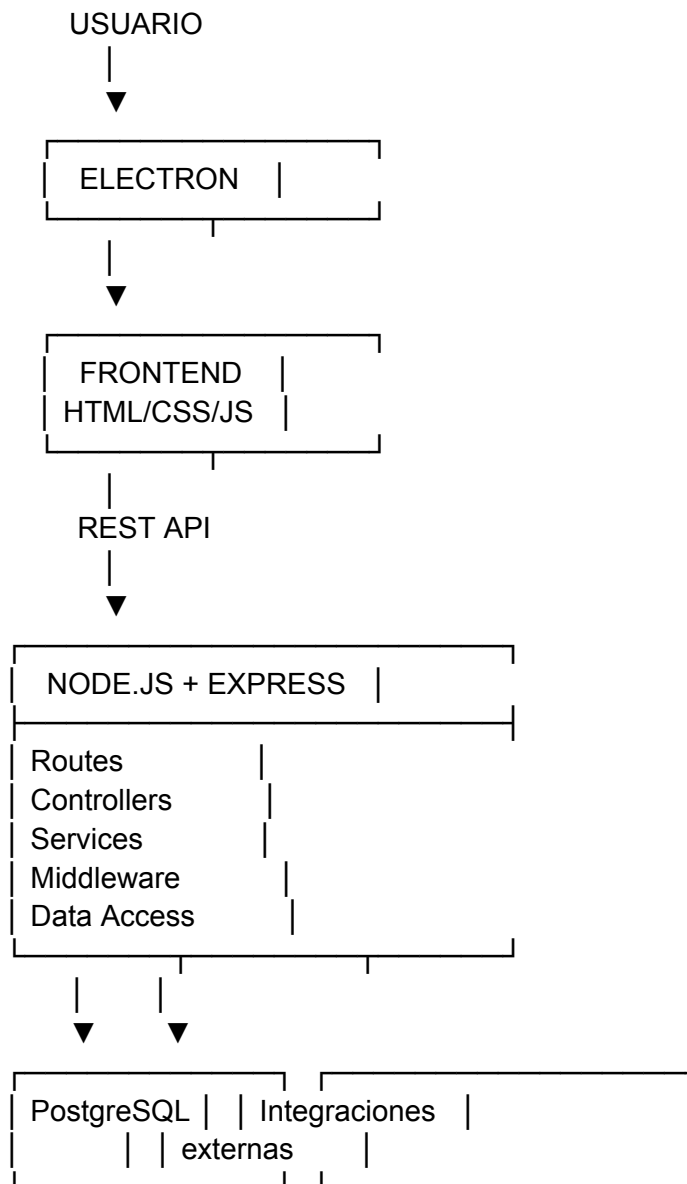
Para el desarrollo se utilizarán:

- PostgreSQL.
- pgAdmin 4.
- Node.js.
- npm.

- Visual Studio Code.
- Git.
- Electron.
- Navegador basado en Chromium para pruebas auxiliares.

Electron será agregado al proyecto mediante npm cuando se cree la estructura inicial.

17. Flujo general del sistema



18. Decisiones arquitectónicas

Se establecen inicialmente las siguientes decisiones:

DA-01. El sistema será una aplicación de escritorio.

DA-02. Electron será utilizado como contenedor de escritorio.

DA-03. El frontend utilizará HTML, CSS y JavaScript.

DA-04. Google Stitch será la referencia de las interfaces previamente diseñadas.

DA-05. Node.js y Express conformarán el backend.

DA-06. La comunicación frontend-backend se realizará mediante una API REST.

DA-07. PostgreSQL será el sistema gestor de base de datos.

DA-08. El frontend no tendrá acceso directo a PostgreSQL.

DA-09. La lógica de negocio estará centralizada en el backend.

DA-10. Las operaciones relevantes deberán poder generar registros de auditoría.

DA-11. Los permisos serán comprobados en el backend según el rol del usuario.

DA-12. Las integraciones externas se implementarán mediante servicios independientes de integración.

DA-13. Las credenciales y secretos de configuración no se almacenarán directamente en el código fuente.

19. Próximas etapas técnicas

Una vez aprobada esta arquitectura, el desarrollo continuará en el siguiente orden:

1. Adaptar el DER existente a PostgreSQL.
2. Diseñar el modelo físico de la base de datos.
3. Definir tipos de datos, claves, restricciones e índices.
4. Crear el script SQL inicial.
5. Crear y validar la base de datos.
6. Crear la estructura inicial del proyecto.
7. Inicializar Node.js y npm.
8. Instalar y configurar Express.
9. Instalar el controlador de PostgreSQL para Node.js.
10. Configurar la conexión Backend–PostgreSQL.
11. Instalar y configurar Electron.
12. Integrar las interfaces basadas en Google Stitch.
13. Implementar autenticación y roles.
14. Implementar reservaciones.

15. Implementar historial y bitácora.
 16. Implementar Daily.
 17. Implementar notificaciones.
 18. Implementar las integraciones externas disponibles.
 19. Ejecutar pruebas funcionales y de integración.
 20. Generar la aplicación de escritorio.
 21. Elaborar documentación técnica y manual de usuario.
-

20. Conclusión

La arquitectura propuesta proporciona una base técnica modular para implementar el Sistema Integrador de Reservaciones.

La combinación de **Electron, HTML/CSS/JavaScript, Node.js, Express y PostgreSQL** permitirá separar la presentación, lógica de negocio y persistencia de información, mientras que la API REST proporcionará un mecanismo uniforme de comunicación entre los componentes.

Esta arquitectura será utilizada como referencia durante el desarrollo. Los cambios estructurales posteriores que afecten tecnologías principales, comunicación entre capas, seguridad, base de datos o integraciones deberán documentarse antes de su implementación.